

Automated Parking Valet

Overview

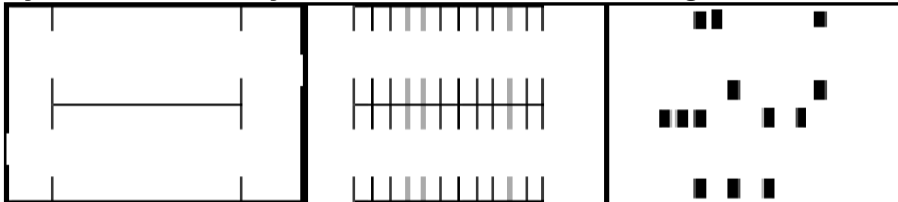
Automatically parking a car that is left in front of a parking lot is a challenging problem. The vehicle's automated systems are expected to take over control and steer the vehicle to an available parking spot. Such a function makes use of multiple on-board sensors. For example:

- Front and side cameras for detecting lane markings, road signs (stop signs, exit markings, etc.), other vehicles, and pedestrians
- Lidar and ultrasound sensors for detecting obstacles and calculating accurate distance measurements
- Ultrasound sensors for obstacle detection
- IMU and wheel encoders for dead reckoning

```
% Set random number generator seed for reproducibility.  
randSeed = rng(0);  
cl = onCleanup(@()rng(randSeed));
```

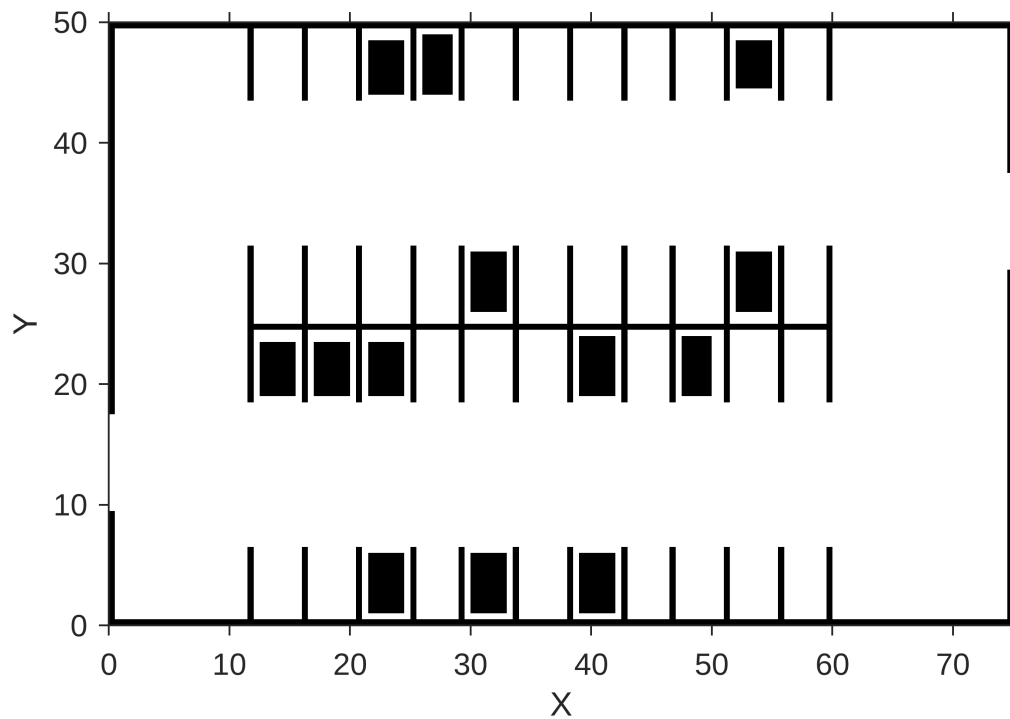
```
mapLayers = loadParkingLotMapLayers;  
plotMapLayers(mapLayers)
```

Map Layers - Stationary Obstacles, Road markings, and Parked Cars



```
% For simplicity, combine the three layers into a single costmap.  
costmap = combineMapLayers(mapLayers);
```

```
figure
plot(costmap, Inflation="off");
```



```
legend off
```

```
costmap.MapExtent % [x, width, y, height] in meters
```

```
ans = 1x4
      0    75     0    50
```

```
costmap.CellSize % cell size in meters
```

```
ans =
0.5000
```

```
vehicleDims      = vehicleDimensions;
maxSteeringAngle = 35; % in degrees
```

```
costmap.CollisionChecker.VehicleDimensions = vehicleDims;
```

```
currentPose = [4 12 0]; % [x, y, theta]
```

```
data = load("routePlan.mat");
routePlan = data.routePlan %#ok<NOPTS>
```

routePlan = 3×3 table

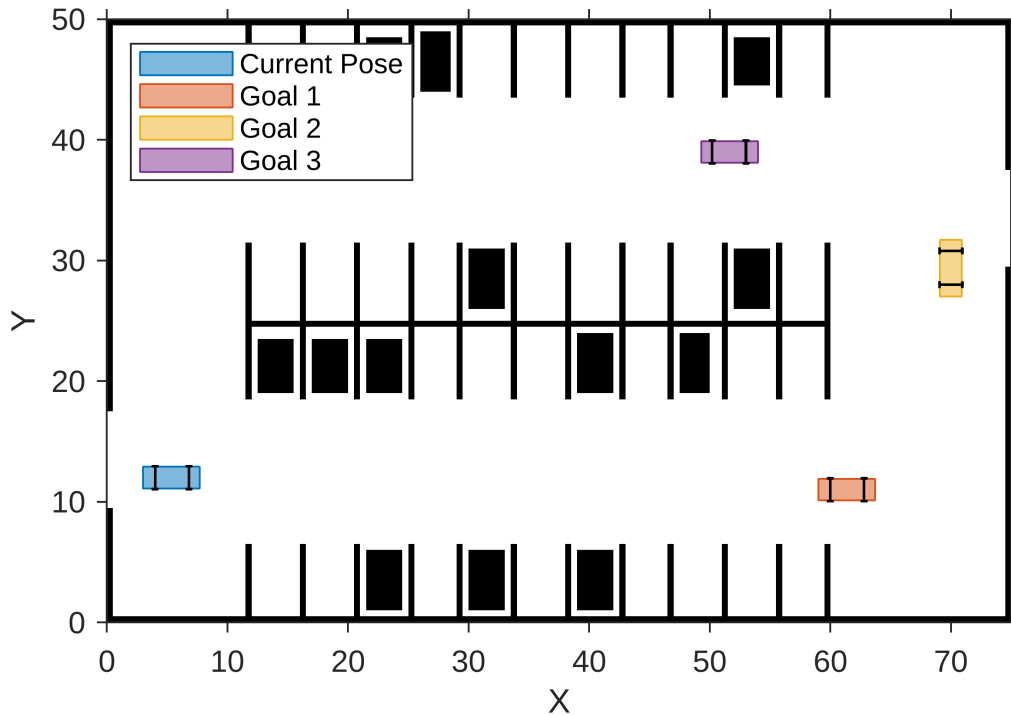
...

	StartPose	
	1	2
1	4	12
2	60	11
3	70	28

```
% Plot vehicle at current pose
hold on
helperPlotVehicle(currentPose, vehicleDims, DisplayName="Current Pose")
legend(Location="northwest")

for n = 1 : height(routePlan)
    % Extract the goal waypoint
    vehiclePose = routePlan{n, "EndPose"};

    % Plot the pose
    legendEntry = "Goal " + n;
    helperPlotVehicle(vehiclePose, vehicleDims, DisplayName=legendEntry);
end
hold off
```



```
behavioralPlanner = HelperBehavioralPlanner(routePlan, maxSteeringAngle);
```

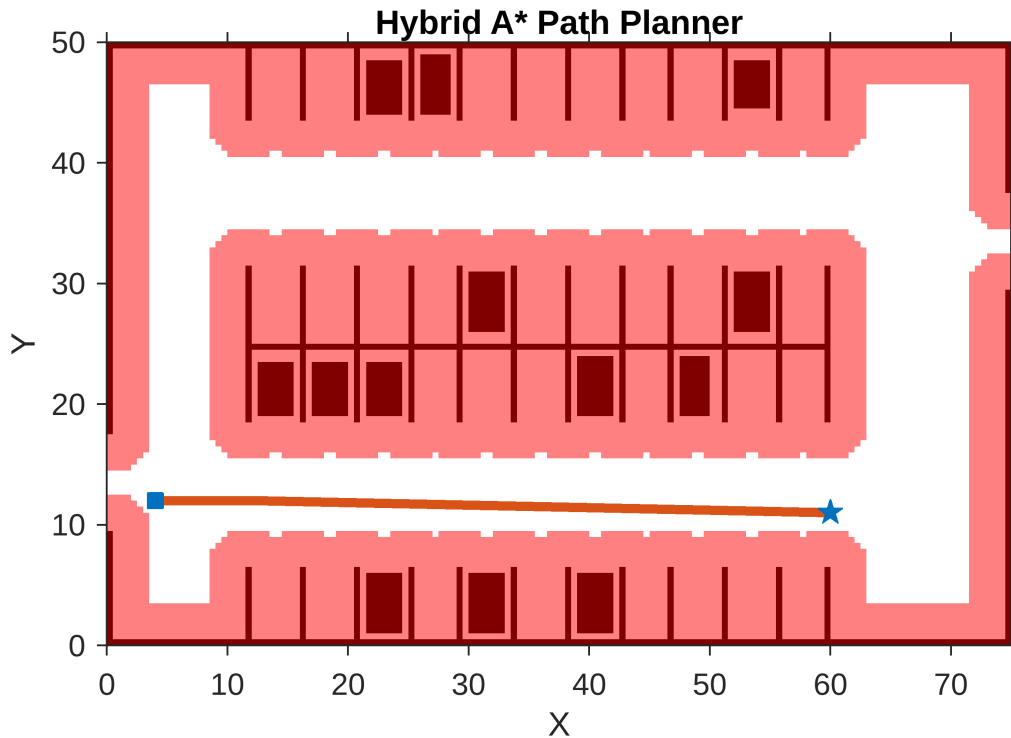
```
ss = stateSpaceSE2;
ss.StateBounds = [costmap.MapExtent(1, 1:2); costmap.MapExtent(1, 3:4); -pi,
pi];
```

```
validator = validatorVehicleCostmap(ss, Map=costmap);
```

```
minTurningRadius = 10; % in meters
motionPlanner = plannerHybridAStar(validator,
MinTurningRadius=minTurningRadius, ...
MotionPrimitiveLength=4); % length in meters
```

```
goalPose = routePlan{1, "EndPose"};
currentPoseRad = [currentPose(1:2) deg2rad(currentPose(3))];
goalPoseRad = [goalPose(1:2) deg2rad(goalPose(3))];
refPath = plan(motionPlanner, currentPoseRad, goalPoseRad);

plannerAxes = show(motionPlanner, Tree="off", HeadingLength=0); % Visualize
the planned path.
```



```
weights = struct(Time=10, Smoothness=100, Obstacle=50);
vehicleInfo = struct("Dimension", [vehicleDims.Length, vehicleDims.Width],
    "Shape", "Rectangle");
```

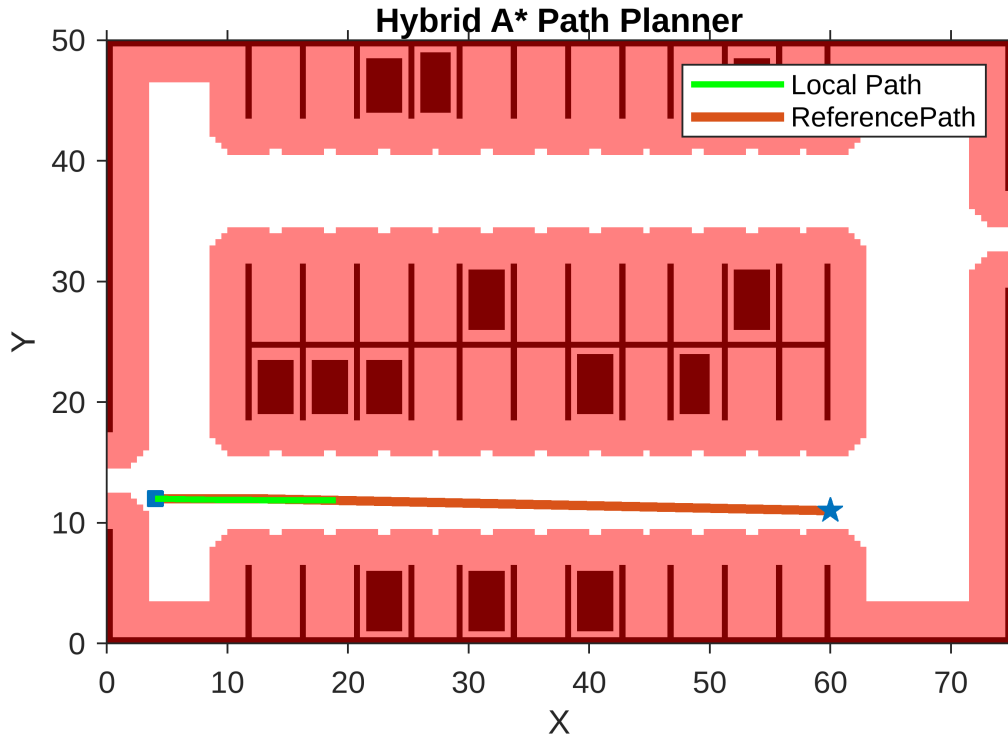
```
localPlanner = controllerTEB(zeros(3,3),...
    MaxVelocity=[5 0.5],... % in m/s and rad/s
    MaxAcceleration=[2 0.5],... % in m/s/s and rad/s/s
    LookAheadTime=3,... % in seconds
    MinTurningRadius=minTurningRadius, NumIteration=2,...
    CostWeights=weights, RobotInformation=vehicleInfo);
```

```
% The local planner does not require knowledge of the full parking lot map
% since it is only planning a short local trajectory. Providing the local
% planner with a smaller map increases planner performance.
maxLocalPlanDistance = 15; % in meters
localPlanner.Map = getLocalMap(costmap, currentPose, maxLocalPlanDistance);
localPlanner.ReferencePath = refPath;
```

```
% Along with an optimized trajectory, |localPath|, the local planner provides
% a reference velocity for each point on the path, |refVel|.
[refVel, ~, localPath, ~] = localPlanner(currentPoseRad, [0 0]);
```

```
% Show the local trajectory alongside the reference path.
```

```
hold(plannerAxes, "on");
plot(plannerAxes, localPath(:,1), localPath(:,2), "g", LineWidth=2,
DisplayName="Local Path");
legend(plannerAxes, "show", findobj(plannerAxes, Type="Line"), {"Local
Path", "ReferencePath"});
```



Vehicle Control and Simulation

The reference speeds, together with the smoothed path, comprise a feasible trajectory that the vehicle can follow. A feedback controller is used to follow this trajectory. The controller corrects errors in tracking the trajectory that arise from tire slippage and other sources of noise, such as inaccuracies in localization. In particular, the controller consists of two components:

- **Lateral control:** Adjust the steering angle such that the vehicle follows the reference path.
- **Longitudinal control:** While following the reference path, maintain the desired speed by controlling the throttle and the brake.

$$\dot{x}_r = v_r * \cos(\theta)$$

$$\dot{y}_r = v_r * \sin(\theta)$$

$$\dot{\theta} = \frac{v_r}{l} * \tan(\delta)$$

$$\dot{v}_r = a_r$$

In the above equations, (x_r, y_r, θ) represents the vehicle pose in world coordinates. v_r , a_r , l , and δ represent the rear-wheel speed, rear-wheel acceleration, wheelbase, and steering angle, respectively. The position and speed of the front wheel can be obtained by:

$$x_f = x_r + l \cos(\theta)$$

$$y_f = y_r + l \sin(\theta)$$

$$v_f = \frac{v_r}{\cos(\delta)}$$

```
closeFigures; % Close all

% Create the vehicle simulator.
vehicleSim = HelperVehicleSimulator(costmap, vehicleDims);

hideFigure(vehicleSim); % Hide vehicle simulation figure

% Configure the simulator to show the trajectory.
vehicleSim.showTrajectory(true);
vehicleSim.showLegend(true);

% Set the vehicle pose and velocity.
vehicleSim.setVehiclePose(currentPose);
currentVel = 0;
vehicleSim.setVehicleVelocity(currentVel);
```

```
pathAnalyzer = HelperPathAnalyzer(Wheelbase=vehicleDims.Wheelbase);
```

```
sampleTime = 0.05;
lonController = HelperLongitudinalController(SampleTime=sampleTime);
```

```
controlRate = HelperFixedRate(1/sampleTime); % in Hertz
```

```
% Set the vehicle pose back to the initial starting point.
currentPose = [4 12 0]; % [x, y, theta]
vehicleSim.setVehiclePose(currentPose);

% Reset velocity.
currentVel = 0; % meters/second
vehicleSim.setVehicleVelocity(currentVel);

% Initialize variables to store vehicle path.
refPath = [];
localPath = [];
```

```

% Setup pathAnalyzer to trigger update of local path every 1 second.
localPlanningFrequency = 1; % 1/seconds
pathAnalyzer.PlanningPeriod = 1/localPlanningFrequency/sampleTime; %
timesteps

isGoalReached = false;

% Initialize count incremented each time the local planner is updated
localPlanCount = 0; % Used for visualization only

showFigure(vehicleSim); % Show vehicle simulation figure.

while ~isGoalReached
    % Plan for the next path segment if near to the next path segment start
    % pose.
    if planNextSegment(behavioralPlanner, currentPose,
2*maxLocalPlanDistance)
        % Request next maneuver from behavioral layer.
        [nextGoal, plannerConfig, speedConfig] =
requestManeuver(behavioralPlanner, ...
                currentPose, currentVel);

        % Plan a reference path using A* planner to the next goal pose.
        if isempty(refPath)
            nextStartRad = [currentPose(1:2) deg2rad(currentPose(3))];
        else
            nextStartRad = refPath(end,:);
        end
        nextGoalRad = [nextGoal(1:2) deg2rad(nextGoal(3))];
        newPath = plan(motionPlanner, nextStartRad, nextGoalRad,
SearchMode="exhaustive");

        % Check if the path is valid. If the planner fails to compute a path,
        % or the path is not collision-free because of updates to the map,
the
        % system needs to re-plan. This scenario uses a static map, so the
path
        % will always be collision-free.
        isReplanNeeded = ~checkPathValidity(newPath.States, costmap);
        if isReplanNeeded
            warning("Unable to find a valid path. Attempting to re-plan.")

            % Request behavioral planner to re-plan
            replanNeeded(behavioralPlanner);
        else
            % Append to refPath
            refPath = [refPath; newPath.States];
            hasNewPath = true;

            vehicleSim.plotReferencePath(refPath); % Plot reference path

```



```

        end
    end

    % Update the local path at the frequency specified by
    % |localPlanningFrequency|
    if pathUpdateNeeded(pathAnalyzer)
        currentPose = getVehiclePose(vehicleSim);
        currentPoseRad = [currentPose(1:2) deg2rad(currentPose(3))];
        currentVel = getVehicleVelocity(vehicleSim);
        currentAngVel = getVehicleAngularVelocity(vehicleSim);

        % Do local planning
        localPlanner.Map = getLocalMap(costmap, currentPose,
maxLocalPlanDistance);

        if hasNewPath
            localPlanner.ReferencePath = refPath;
            hasNewPath = false;
        end

        [localVel, ~, localPath, ~] = localPlanner(currentPoseRad,
[currentVel currentAngVel]);

        vehicleSim.plotLocalPath(localPath); % Plot new local path

        % For visualization only
        if mod(localPlanCount, 20) == 0
            snapnow; % Capture state of the figures
        end
        localPlanCount = localPlanCount+1;

        % Configure path analyzer.
        pathAnalyzer.RefPoses = [localPath(:,1:2)
rad2deg(localPath(:,3))];
        pathAnalyzer.Directions = sign(localVel(:,1));
        pathAnalyzer.VelocityProfile = localVel(:,1);
    end

    % Find the reference pose on the path and the corresponding
    % velocity.
    [refPose, refVel, direction] = pathAnalyzer(currentPose, currentVel);

    % Update driving direction for the simulator.
    updateDrivingDirection(vehicleSim, direction);

    % Compute steering command.
    steeringAngle = lateralControllerStanley(refPose, currentPose,
currentVel, ...
        Direction=direction, Wheelbase=vehicleDims.Wheelbase,
PositionGain=4);

```

```

% Compute acceleration and deceleration commands.
lonController.Direction = direction;
[accelCmd, decelCmd] = lonController(refVel, currentVel);

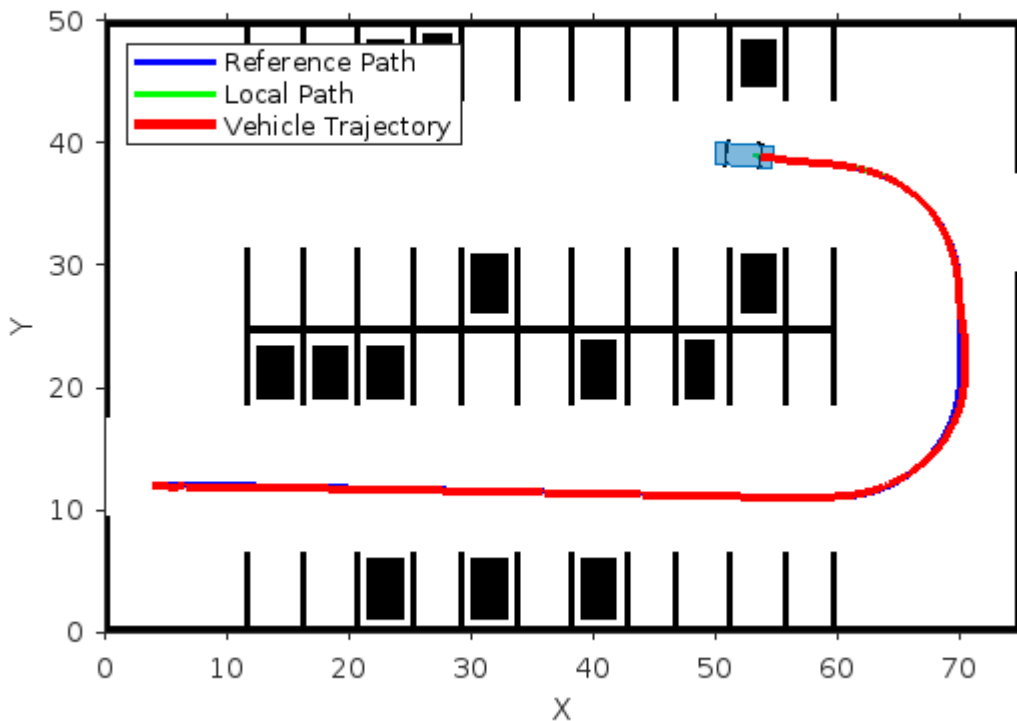
% Simulate the vehicle using the controller outputs.
drive(vehicleSim, accelCmd, decelCmd, steeringAngle);

% Get current pose and velocity of the vehicle.
currentPose = getVehiclePose(vehicleSim);
currentVel = getVehicleVelocity(vehicleSim);

% Check if the vehicle reaches the goal.
isGoalReached = helperGoalChecker(nextGoal, currentPose, currentVel,
speedConfig.EndSpeed, direction);

% Wait for fixed-rate execution.
waitfor(controlRate);
end

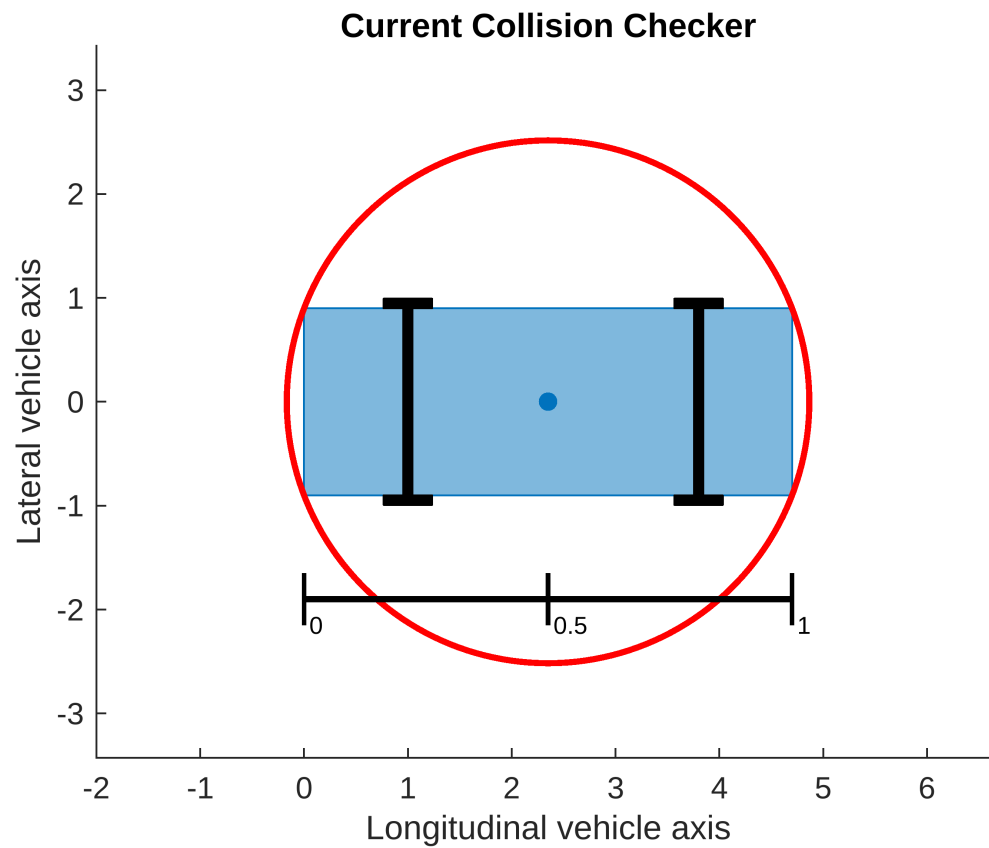
```



```
hideFigure(vehicleSim); % Hide vehicle simulation figure
```

```
ccConfig = costmap.CollisionChecker;
```

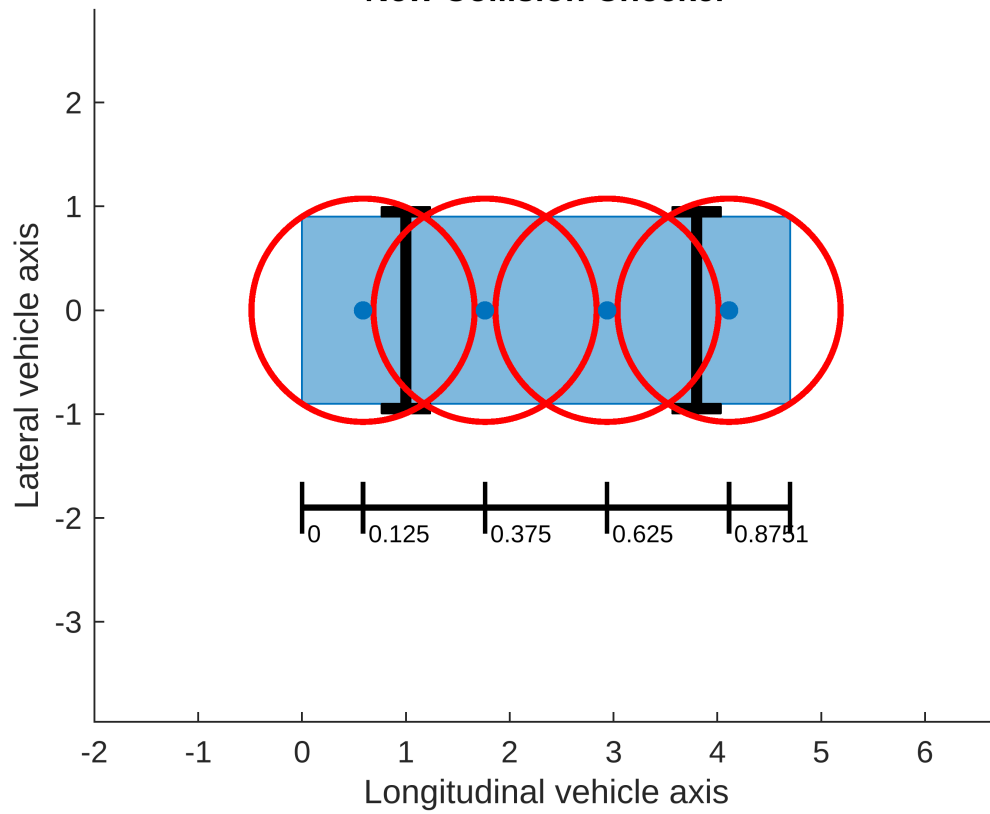
```
figure
plot(ccConfig)
title("Current Collision Checker")
```



```
ccConfig.NumCircles = 4;

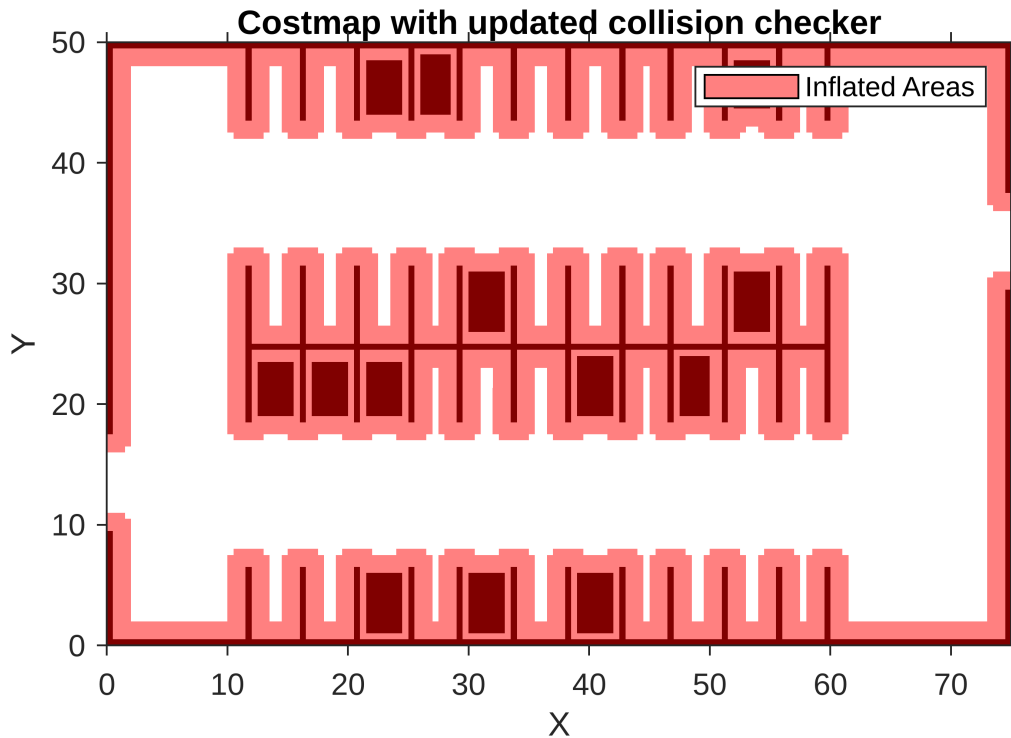
figure
plot(ccConfig)
title("New Collision Checker")
```

New Collision Checker



```
costmap.CollisionChecker = ccConfig;
```

```
figure  
plot(costmap)  
title("Costmap with updated collision checker")
```



```
% Set up the pathPlannerRRT to use the updated costmap.
parkMotionPlanner = pathPlannerRRT(costmap, MinIterations=1000);

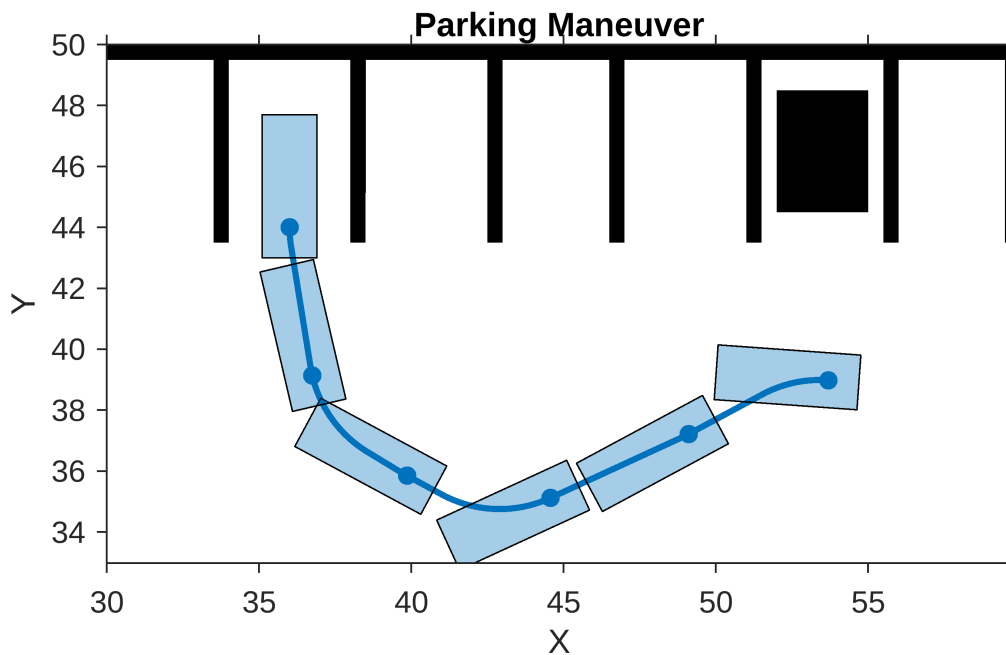
% Define desired pose for the parking spot, returned by the V2X system.
parkPose = [36 44 90];
preParkPose = currentPose;

% Compute the parking maneuver.
replan = true;
while replan
    % Compute the required parking maneuver.
    refPath = plan(parkMotionPlanner, preParkPose, parkPose);

    % Retrieve transition poses and directions from the planned path.
    [transitionPoses, directions] = interpolate(refPath);

    replan = isempty(refPath.PathSegments) || any(directions == -1) ||
    refPath.Length > 30;
end

% Plot the resulting parking maneuver.
figure
plotParkingManeuver(costmap, refPath, preParkPose, parkPose)
```



```
% Specify number of poses to return using a separation of approximately 1 m.
approxSeparation = 1; % meters
```

```
% Smooth the path.
```

```
numSmoothPoses = round(refPath.Length / approxSeparation);
[refPoses, directions, cumLengths, curvatures] =
smoothPathSpline(transitionPoses, directions, numSmoothPoses);
```

```
% Set up the velocity profile generator to stop at the end of the trajectory,
% with a speed limit of 5 mph.
```

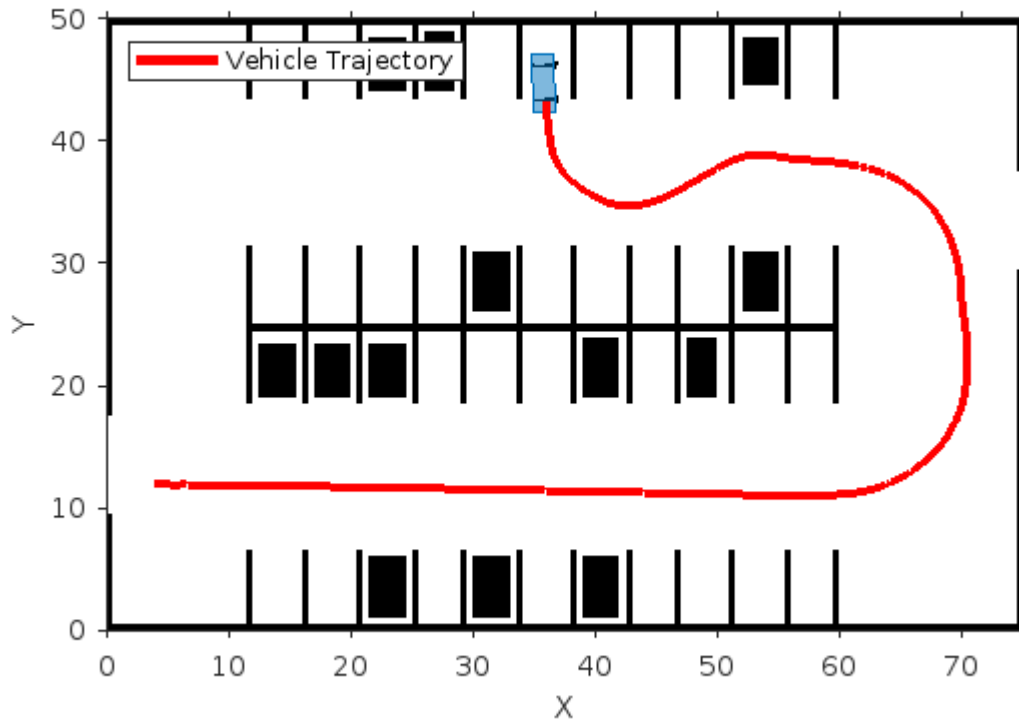
```
refVelocities = helperGenerateVelocityProfile(directions, cumLengths,
curvatures, currentVel, 0, 2.2352);
```

```
showFigure(vehicleSim); % Show vehicle simulation figure.
```

```
pathAnalyzer.RefPoses = refPoses;
pathAnalyzer.Directions = directions;
pathAnalyzer.VelocityProfile = refVelocities;
```

```
% Clear old path planning data from the plot
```

```
vehicleSim.clearReferencePath();
vehicleSim.clearLocalPath();
```



```
% Reset longitudinal controller.
reset(lonController);

isGoalReached = false;

while ~isGoalReached
    % Find the reference pose on the path and the corresponding velocity.
    [refPose, refVel, direction] = pathAnalyzer(currentPose, currentVel);

    % Update driving direction for the simulator.
    updateDrivingDirection(vehicleSim, direction);

    % Compute steering command.
    steeringAngle = lateralControllerStanley(refPose, currentPose,
currentVel, ...
        Direction=direction, Wheelbase=vehicleDims.Wheelbase);

    % Compute acceleration and deceleration commands.
    lonController.Direction = direction;
    [accelCmd, decelCmd] = lonController(refVel, currentVel);

    % Simulate the vehicle using the controller outputs.
    drive(vehicleSim, accelCmd, decelCmd, steeringAngle);
```

```

    % Check if the vehicle reaches the goal.
    isGoalReached = helperGoalChecker(parkPose, currentPose, currentVel, 0,
direction);

    % Wait for fixed-rate execution.
    waitfor(controlRate);

    % Get current pose and velocity of the vehicle.
    currentPose = getVehiclePose(vehicleSim);
    currentVel = getVehicleVelocity(vehicleSim);
end

closeFigures; % Close all

```

```

% Specify a parking pose corresponding to a back-in parking maneuver.
parkPose = [49 47.2 -90];

% Change the connection method to allow for reverse motions.
parkMotionPlanner.ConnectionMethod = "Reeds-Shepp";

```

```

parkMotionPlanner.MinTurningRadius = 12; % meters
parkMotionPlanner.ConnectionDistance = 15; % meters

% Reset vehicle pose and velocity.
currentVel = 0; % m/s
vehicleSim.setVehiclePose(preParkPose);
vehicleSim.setVehicleVelocity(currentVel);

% Compute the parking maneuver.
replan = true;
while replan
    refPath = plan(parkMotionPlanner, preParkPose, parkPose);

    % The path corresponding to the parking maneuver is small and requires
    % precise maneuvering. Instead of interpolating only at transition poses,
    % interpolate more finely along the length of the path.

    numSamples = 10;
    stepSize = refPath.Length / numSamples;
    lengths = 0 : stepSize : refPath.Length;

    [transitionPoses, directions] = interpolate(refPath, lengths);

    % Replan if the path contains more than one direction switching poses
    % or if the path is too long.
    replan = isempty(refPath.PathSegments) ||
sum(abs(diff(directions(1:end-1))))~=2 || refPath.Length > 25;
end

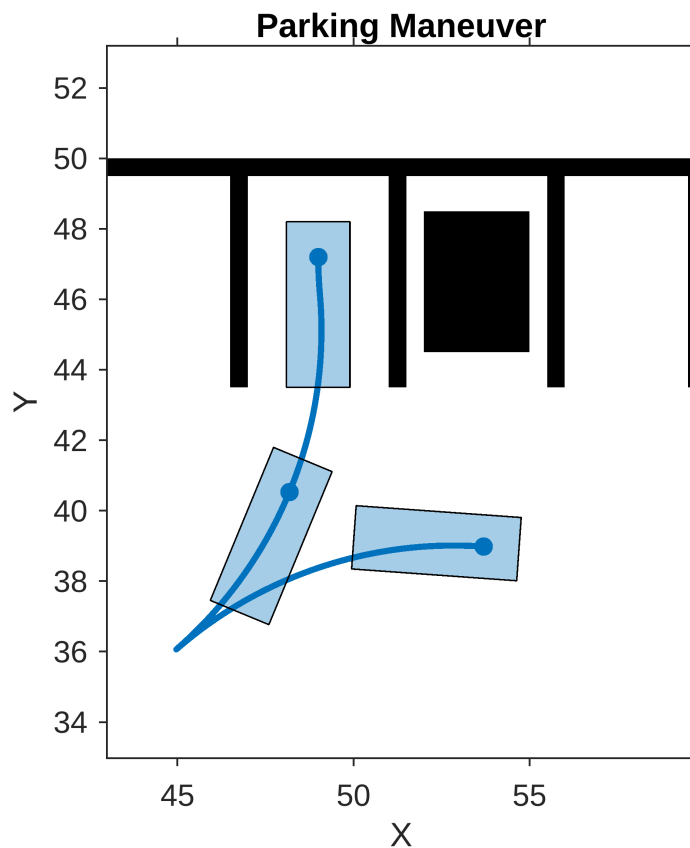
```



```
% Visualize the parking maneuver.
```

```
figure
```

```
plotParkingManeuver(costmap, refPath, preParkPose, parkPose)
```



```
% Smooth the path.
```

```
numSmoothPoses = round(refPath.Length / approxSeparation);
```

```
minSeparation = 0.5;
```

```
[refPoses, directions, cumLengths, curvatures] =
```

```
smoothPathSpline(transitionPoses, directions, numSmoothPoses, minSeparation);
```

```
% Set up the velocity profile generator to stop at the end of the trajectory,  
% with a speed limit of 5 mph.
```

```
refVelocities = helperGenerateVelocityProfile(directions, cumLengths,  
curvatures, currentVel, 0, 2.2352);
```

Simulate reverse parking maneuver.

```
pathAnalyzer.RefPoses = refPoses;
```

```
pathAnalyzer.Directions = directions;
```

```
pathAnalyzer.VelocityProfile = refVelocities;
```

```
reset(lonController); % Reset longitudinal controller.
```

```
isGoalReached = false;
```

```

while ~isGoalReached
    % Get current driving direction.
    currentDir = getDrivingDirection(vehicleSim);

    % Find the reference pose on the path and the corresponding velocity.
    [refPose, refVel, direction] = pathAnalyzer(currentPose, currentVel);

    % If the vehicle changes driving direction, reset vehicle velocity in
    % the simulator and reset longitudinal controller.
    if currentDir ~= direction
        currentVel = 0;
        setVehicleVelocity(vehicleSim, currentVel);
        reset(lonController);
    end

    % Update driving direction for the simulator. If the vehicle changes
    % driving direction, reset and return the current vehicle velocity as
    zero.
    currentVel = updateDrivingDirection(vehicleSim, direction, currentDir);

    % Compute steering command.
    steeringAngle = lateralControllerStanley(refPose, currentPose,
currentVel, ...
        Direction=direction, Wheelbase=vehicleDims.Wheelbase);

    % Compute acceleration and deceleration commands.
    lonController.Direction = direction;
    [accelCmd, decelCmd] = lonController(refVel, currentVel);

    % Simulate the vehicle using the controller outputs.
    drive(vehicleSim, accelCmd, decelCmd, steeringAngle);

    % Check if the vehicle reaches the goal.
    isGoalReached = helperGoalChecker(parkPose, currentPose, currentVel, 0,
direction);

    % Wait for fixed-rate execution.
    waitfor(controlRate);

    % Get current pose and velocity of the vehicle.
    currentPose = getVehiclePose(vehicleSim);
    currentVel = getVehicleVelocity(vehicleSim);
end

closeFigures;
snapnow; % Capture state of the simulation figure

delete(vehicleSim); % Delete the simulator.

```

References

[1] Buehler, Martin, Karl Iagnemma, and Sanjiv Singh. *The DARPA Urban Challenge: Autonomous Vehicles in City Traffic* (1st ed.). Springer Publishing Company, Incorporated, 2009.

[2] Lepetic, Marko, Gregor Klančar, Igor Skrjanc, Drago Matko, Bostjan Potocnik, "Time Optimal Path Planning Considering Acceleration Limits." *Robotics and Autonomous Systems*. Volume 45, Issues 3-4, 2003, pp. 199-210.

Supporting Functions

loadParkingLotMapLayers Load environment map layers for parking lot

```
function mapLayers = loadParkingLotMapLayers()
% Load occupancy maps corresponding to 3 layers - obstacles, road
% markings, and used spots.

mapLayers.StationaryObstacles = imread("stationary.bmp");
mapLayers.RoadMarkings       = imread("road_markings.bmp");
mapLayers.ParkedCars         = imread("parked_cars.bmp");
end
```

plotMapLayers Plot struct containing map layers

```
function plotMapLayers(mapLayers)
% Plot the multiple map layers on a figure window.

figure
cellofMaps = cellfun(@imcomplement, struct2cell(mapLayers),
UniformOutput=false);
montage(cellofMaps, Size=[1 numel(cellofMaps)], Border=[5 5],
ThumbnailSize=[300 NaN])
title("Map Layers - Stationary Obstacles, Road markings, and Parked Cars")
end
```

combineMapLayers Combine map layers into a single costmap

```
function costmap = combineMapLayers(mapLayers)
% Combine map layers struct into a single vehicleCostmap.

combinedMap = mapLayers.StationaryObstacles + mapLayers.RoadMarkings + ...
    mapLayers.ParkedCars;
combinedMap = im2single(combinedMap);

costmap = vehicleCostmap(combinedMap, CellSize=0.5);
end
```

getLocalMap Extract area local to vehicle from costmap

```
function localMap = getLocalMap(costmap, pose, maxDistance)
% Extract area within maxDistance of pose from costmap
```

```

mapExtent = costmap.MapExtent;
cellSize = costmap.CellSize;
xMin = max(pose(1)-maxDistance, mapExtent(1));
xMax = min(pose(1)+maxDistance, mapExtent(2));
yMin = max(pose(2)-maxDistance, mapExtent(3));
yMax = min(pose(2)+maxDistance, mapExtent(4));

xMinCell = round(xMin/cellSize)+1;
xMaxCell = round(xMax/cellSize);
yMaxCell = costmap.MapSize(1) - round(yMin/cellSize);
yMinCell = costmap.MapSize(1) - round(yMax/cellSize)+1;

costs = getCosts(costmap);
localCosts = costs(yMinCell:yMaxCell, xMinCell:xMaxCell);
localMap = occupancyMap(localCosts, 1/cellSize);
localMap.GridLocationInWorld = [xMin yMin];
end

```

plotVelocityProfile Plot speed profile

```

function plotVelocityProfile(path, refVelocities, maxSpeed)
% Plot the generated velocity profile.

% Calculate cumulative path lengths
cumPathLength = pdist(path(:, 1:2));

% Plot reference velocity along length of the path.
plot(cumPathLength, refVelocities, LineWidth=2);

% Plot a line to display maximum speed.
hold on
line([0;cumPathLength(end)], [maxSpeed;maxSpeed], Color="r")
hold off

% Set axes limits.
buffer = 2;
xlim([0 cumPathLength(end)]);
ylim([0 maxSpeed + buffer])

% Add labels.
xlabel("Cumulative Path Length (m)");
ylabel("Velocity (m/s)");

% Add legend and title.
legend("Velocity Profile", "Max Speed")
title("Generated velocity profile")
end

```

closeFigures

```

function closeFigures()
% Close all the figures except the simulator visualization.

% Find all the figure objects.
figHandles = findobj(Type="figure");
for i = 1: length(figHandles)
    if ~strcmp(figHandles(i).Name, "Automated Valet Parking")
        close(figHandles(i));
    end
end
end

```

plotParkingManeuver Display the generated parking maneuver on a costmap.

```

function plotParkingManeuver(costmap, refPath, currentPose, parkPose)
% Plot the generated parking maneuver on a costmap.

% Plot the costmap, without inflated areas.
plot(costmap, Inflation="off")

% Plot reference parking maneuver on the costmap.
hold on
plot(refPath, DisplayName="Parking Maneuver")

title("Parking Maneuver")

% Zoom into parking maneuver by setting axes limits.
lo = min([currentPose(1:2); parkPose(1:2)]);
hi = max([currentPose(1:2); parkPose(1:2)]);

buffer = 6; % meters

xlim([lo(1)-buffer hi(1)+buffer])
ylim([lo(2)-buffer hi(2)+buffer])
end

```